

Name : P. Yaswanth

Reg No 192524167

Experiment: Map Coloring Problem using Constraint Satisfaction Problem (CSP)

Aim

To implement the **Map Coloring Problem** using Python based on the **Constraint Satisfaction Problem (CSP)** approach.

Objectives

- To understand the concept of Constraint Satisfaction Problems (CSP).
- To implement the Map Coloring Problem using Python.
- To assign colors to regions such that no two adjacent regions have the same color.
- To apply the backtracking algorithm to satisfy all constraints.
- To understand the applications of CSP in Artificial Intelligence.

Algorithm

1. Represent the map as a graph where each region is a node.
2. Define a set of available colors.
3. Select an uncolored region.
4. Assign a color that does not conflict with its neighboring regions.
5. If no valid color is available, backtrack and try another color.
6. Repeat until all regions are assigned valid colors.
7. Display the final color assignment.

Python Program

Map Coloring using CSP (Backtracking)

```
graph = {  
    'A': ['B', 'C', 'D'],  
    'B': ['A', 'C'],
```

```
'C': ['A', 'B', 'D'],  
'D': ['A', 'C']  
}
```

```
colors = ['Red', 'Green', 'Blue']
```

```
result = {}
```

```
def is_safe(node, color):  
    for neighbor in graph[node]:  
        if neighbor in result and result[neighbor] == color:  
            return False  
    return True
```

```
def solve():  
    if len(result) == len(graph):  
        return True
```

```
for node in graph:  
    if node not in result:  
        for color in colors:  
            if is_safe(node, color):  
                result[node] = color  
  
        if solve():  
            return True
```

```
        del result[node]
    return False
```

```
solve()
```

```
print("Map Coloring Solution:")
```

```
for region in result:
```

```
    print(region, "->", result[region])
```

Sample Output

Map Coloring Solution:

A -> Red

B -> Green

C -> Blue

D -> Green

Out Put

```
1 # Map Coloring using CSP (Backtracking)
2
3 graph = {
4     'A': ['B', 'C', 'D'],
5     'B': ['A', 'C'],
6     'C': ['A', 'B', 'D'],
7     'D': ['A', 'C']
8 }
9
10 colors = ['Red', 'Green', 'Blue']
11
12 result = {}
13
14 def is_safe(node, color):
15     for neighbor in graph[node]:
16         if neighbor in result and result[neighbor] == color:
17             return False
18     return True
19
20 def solve():
21     if len(result) == len(graph):
22         return True
23
24     for node in graph:
25         if node not in result:
26             for color in colors:
```

Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Green

=== Code Execution Successful ===

Result

The Map Coloring Problem was successfully implemented using the **Constraint Satisfaction Problem (CSP)** approach in Python. The program assigned colors to all regions such that no two adjacent regions had the same color.

Conclusion

The experiment demonstrated the implementation of the **Map Coloring Problem** using the **backtracking algorithm**, a common technique for solving Constraint Satisfaction Problems (CSPs). The program successfully assigned valid colors while satisfying all adjacency constraints. This experiment illustrates the practical application of CSPs in Artificial Intelligence for solving scheduling, planning, and resource allocation problems.